SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:		Reg. No.:	
Date:		Duration:	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – CI/CD Pipeline Design Exercise

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design a CI/CD (Continuous Integration and Continuous Deployment) pipeline** for the proposed software system to automate the processes of code integration, building, testing, and deployment.

The exercise should include:

1. **Analyze the project requirements** and identify the CI/CD needs of the assigned problem statement.
2. **Design the CI/CD pipeline workflow** showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.
3. Identify suitable **tools and technologies** for each stage of the pipeline and justify their selection.
4. Define appropriate **automated testing strategies** to be integrated into the pipeline.
5. Design the **deployment strategy**, including development, testing/staging, and production environments.
6. Incorporate appropriate **security, quality checks, notifications, and rollback mechanisms** in the pipeline.
7. Prepare a **CI/CD pipeline diagram** and explain the workflow from code commit to deployment.

Deliverable: Submit a **CI/CD Pipeline Design document** containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25 %)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration(15 %)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate
Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollback details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation (10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/documentation but lacks clarity	Poorly presented, incomplete, or difficult to understand

Criteria	Mark
Requirement & Pipeline Analysis(15%)	/5
Pipeline Architecture & Workflow(25%)	/4
Tools & Technology Selection(20%)	/4
Automated Testing & Quality Integration(15%)	/4
Deployment, Security & Rollback Strategy(15%)	/4
Pipeline Diagram & Documentation(10%)	/4
TOTAL	/25

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD PIPELINE DESIGN EXERCISE

CI/CD Pipeline Design Document

Smart Classroom Attendance, Engagement, and Performance Analytics System

Course Code	CSA10
Course Title	Software Engineering
CO Assigned	CO4
Assessment	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage	20%
Total Marks	25
Duration	60 minutes
Student Name	SHAIK ARSHAD
Register No.	192425080
Date	19-08-2026

Code of Conduct

*I **SHAIK ARSHAD(192425080)** (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: ARSHAD

1. Problem Statement & Project Overview

Educational institutions currently rely on manual attendance registers and periodic, subjective evaluation of student performance. This is time-consuming, delays intervention for at-risk students, and gives instructors little visibility into real-time classroom engagement. The proposed Smart Classroom Attendance, Engagement, and Performance Analytics System automates attendance capture through AI-based facial recognition or RFID, continuously analyzes participation and academic performance, and uses predictive analytics to flag at-risk students early, enabling personalized interventions.

Because the system directly affects daily classroom operations and handles sensitive student data (biometric identifiers, attendance records, academic performance), it must be released to production through a rigorous, automated CI/CD pipeline that enforces code quality, security, and reliability at every stage before any change reaches real classrooms.

1.1 Project Scope

The scope of this design exercise covers the CI/CD pipeline for four core services that make up the platform: the Attendance Capture Service (facial recognition / RFID ingestion), the Engagement Analytics Service (participation scoring), the Performance Prediction Service (at-risk student modeling), and the Instructor Dashboard (web front end). The pipeline design is technology-agnostic in principle but is illustrated here with a representative, industry-standard toolchain that a college or university IT team could realistically adopt.

1.2 Key Stakeholders

- Instructors and academic coordinators – primary consumers of attendance, engagement, and at-risk alerts.
- Students – subjects of attendance and performance data; require privacy protection.
- Institution IT / DevOps team – owns the pipeline, environments, and on-call rollback response.
- Academic administration – uses aggregated analytics dashboards for institutional decision-making.
- Compliance / data-protection officer – reviews security gates and audit logs for regulatory adherence.

1.3 Assumptions & Constraints

- The institution has classroom hardware capable of capturing facial images or RFID taps and forwarding them to the platform over campus network / Wi-Fi.
- Releases must not disrupt attendance capture during active class hours, so deployments favor zero-downtime strategies.
- All student data is subject to institutional data-protection policy and, where applicable, national privacy regulations.
- The pipeline assumes a cloud or on-premise Kubernetes-capable environment is available for container orchestration.

2. Requirement & Pipeline Analysis

2.1 CI/CD Needs Derived from the Problem Statement

- Frequent, safe releases: attendance, engagement-analysis, and prediction modules will be updated independently, so the pipeline must support modular, service-level builds and deployments.

- High reliability: attendance and analytics must be available during live class hours; the pipeline must validate builds thoroughly before they reach production and support instant rollback.
- Data protection & compliance: facial recognition data and student academic records are sensitive, so every build must pass security and privacy checks (secrets scanning, dependency vulnerability scanning, access-control tests) before deployment.
- Model-driven components: the at-risk prediction model needs its own validation stage (accuracy/drift checks) in addition to conventional software tests.
- Multi-environment rollout: changes should be verified in isolated development and staging/QA environments that mirror production before being released to live classrooms.
- Auditability: all changes must be traceable (commit → build → test → deployment) to support academic-institution compliance and incident investigation.

2.2 Non-Functional Requirements Driving Pipeline Design

Quality Attribute	Pipeline Implication
Availability	Zero/low-downtime deployment strategy (blue-green or canary) and automated health-check-triggered rollback.
Security & Privacy	Mandatory secrets scanning, dependency scanning, and access-control tests as hard gates before packaging.
Accuracy	Dedicated model-validation stage for the prediction service with accuracy/drift thresholds.
Scalability	Containerized, horizontally scalable services orchestrated by Kubernetes to handle peak class-change load.
Auditability	Immutable, versioned artifacts and pipeline logs retained for institutional compliance review.

3. System Architecture Overview

Before describing the pipeline itself, it is useful to summarize the runtime architecture the pipeline builds and deploys. The platform follows a microservice architecture so that each functional area — attendance, engagement, prediction, and reporting — can be built, tested, and released independently, which is precisely what a CI/CD pipeline is designed to support.

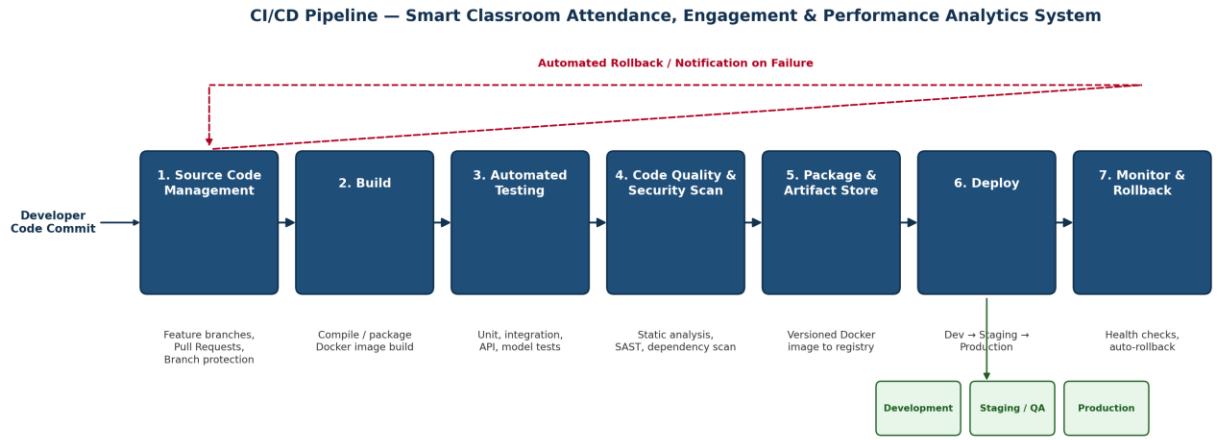
Component	Responsibility
Attendance Capture Service	Ingests facial-recognition or RFID events from classroom devices and records attendance in near real time.
Engagement Analytics Service	Processes participation signals (question responses, activity logs) into an engagement score per student per session.
Performance Prediction Service	Machine-learning service that combines attendance, engagement, and grade history to flag at-risk students.
Instructor Dashboard	Web front end presenting attendance, engagement trends, and predictive alerts to instructors and coordinators.

Component	Responsibility
Shared Data Layer	Encrypted relational + analytics data store shared by all services via versioned APIs.

Each of these components is packaged as an independent container image and moves through the same seven-stage pipeline described in Section 4, allowing the team to release, for example, an improved prediction model without redeploying the attendance-capture service.

4. Pipeline Architecture & Workflow

The pipeline is triggered automatically on every commit and pull request, and executes seven sequential stages, with an automated rollback/notification loop feeding back to the development team on any failure.



4.1 Stage-by-Stage Workflow

- Source Code Management – Developers work on feature branches (e.g., feature/attendance-rfid, feature/engagement-model). Pull requests require at least one peer review and passing status checks before merge into the main branch; branch protection rules block direct pushes to main.
- Build – On every push, the pipeline compiles the affected microservice (attendance service, engagement-analytics service, prediction service, dashboard front end) and builds a versioned Docker image tagged with the commit SHA.
- Automated Testing – Unit tests, integration tests (API contracts between attendance, analytics, and dashboard services), and, for the prediction module, model-validation tests run automatically; the build fails fast if coverage or accuracy thresholds are not met.
- Code Quality & Security Analysis – Static code analysis, dependency vulnerability scanning, and secrets detection run against the codebase; any critical issue blocks the pipeline from proceeding to packaging.
- Packaging – Validated builds are packaged as immutable, versioned container images and pushed to a private artifact/container registry, ready for deployment.

- Deployment – Images are promoted through Development → Staging/QA → Production environments; production deployment uses a controlled strategy (blue-green or canary) to avoid classroom downtime.
- Monitoring & Rollback – Post-deployment health checks and real-time monitoring watch key metrics (API latency, attendance-capture accuracy, error rate); if thresholds are breached, the pipeline automatically rolls back to the last stable release and notifies the DevOps/instructor-support team.

4.2 Pipeline Triggers

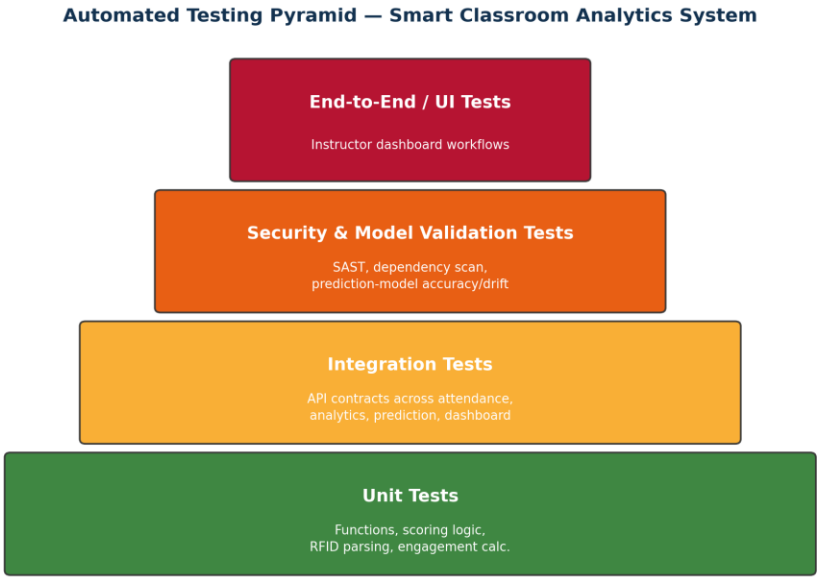
- Push / pull-request trigger: any commit to a feature branch or PR against main runs stages 1–4 (source, build, test, quality/security) to give the developer fast feedback.
- Merge trigger: a merge into main additionally runs packaging and deploys to the Development environment automatically.
- Manual approval gate: promotion from Staging/QA to Production requires an explicit approval from a designated release owner, in addition to all automated checks passing.
- Scheduled trigger: nightly pipeline run against main performs a full regression suite and dependency-vulnerability re-scan to catch newly disclosed CVEs in existing dependencies.

5. Tools & Technology Selection

Pipeline Stage	Recommended Tool(s)	Justification
Source Code Management	Git + GitHub / GitLab	Industry-standard version control with built-in pull-request review, branch protection, and webhook triggers for CI.
CI/CD Orchestration	GitHub Actions / Jenkins	Native integration with the Git host, YAML-defined pipelines, and wide plugin ecosystem for multi-service builds.
Build & Packaging	Docker, Maven/Gradle or pip/Poetry	Containerization ensures consistent runtime across dev, staging, and production; language build tools handle dependency resolution.
Automated Testing	JUnit/PyTest, Postman/Newman, MLflow (model tests)	Covers unit, API/integration, and ML-model accuracy validation needed for the prediction module.
Code Quality & Security	SonarQube, Snyk / OWASP Dependency-Check, GitLeaks	Detects code smells, known CVEs in dependencies, and hard-coded secrets — critical given the system handles biometric data.
Artifact Registry	Docker Hub / Amazon ECR	Central, versioned storage for build artifacts, enabling rollback to any prior stable image.
Deployment & Orchestration	Kubernetes + Helm, or AWS ECS	Automates multi-environment rollout, supports blue-green/canary strategies, and self-heals failed pods.
Monitoring & Alerting	Prometheus + Grafana, Slack/Email alerts	Provides real-time dashboards on system health and instant notifications to trigger rollback decisions.

6. Automated Testing Strategy

Testing is organized as a pyramid: a large base of fast unit tests, a smaller layer of integration tests, a focused layer of security and model-validation tests, and a thin top layer of end-to-end UI tests. This keeps feedback fast for developers while still catching integration, security, and model-quality issues before release.



6.1 Test Layers in Detail

- Unit tests: validate individual functions such as face-match confidence scoring, RFID event parsing, and engagement-score calculation; run on every commit with a minimum coverage gate (e.g., 80%).
- Integration tests: verify API contracts between the attendance service, engagement-analytics service, prediction service, and dashboard, using containerized test environments spun up in CI.
- Model validation tests: the at-risk-prediction model is evaluated against a held-out dataset for accuracy, precision/recall, and drift; deployment is blocked if performance falls below the agreed threshold.
- Security tests: dependency and container image scans, plus basic penetration checks on authentication/authorization endpoints, run before packaging.
- End-to-end / UI tests: automated browser tests validate the instructor dashboard workflow (viewing attendance, engagement trends, and at-risk alerts) in the staging environment prior to production release.

6.2 Test Data Strategy

- Synthetic and anonymized datasets are used in CI and Staging/QA so that no real student data is exposed to automated test environments.
- A fixed “golden dataset” of labeled attendance and performance records is used to benchmark the prediction model's accuracy on every build, so regressions are caught immediately.
- Test data is refreshed periodically from a de-identified snapshot of production data, governed by the institution's data-protection policy.

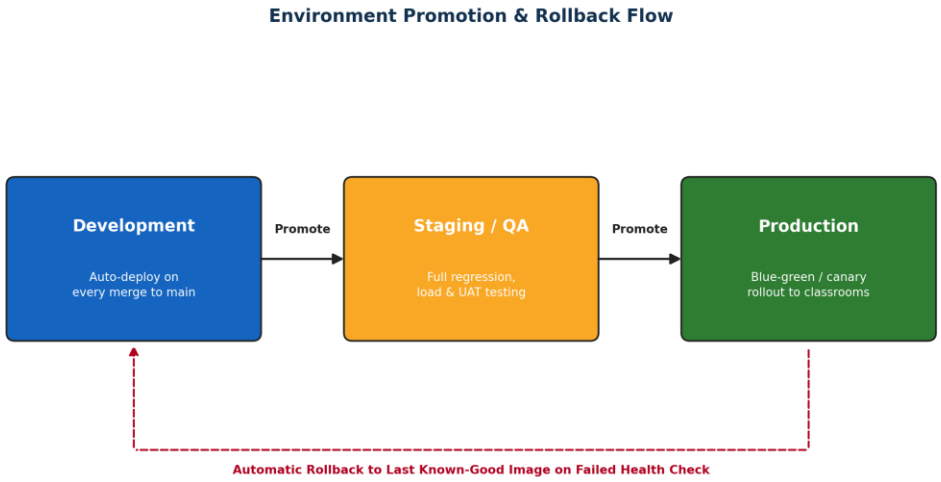
6.3 Quality Gates

Gate	Pass Criteria
Unit test coverage	≥ 80% line coverage on changed modules; zero failing tests.
Integration tests	All contract tests between services pass against the current API schema.

Gate	Pass Criteria
Static analysis	No new critical or high-severity code-quality issues introduced.
Dependency / secrets scan	Zero critical CVEs and zero exposed secrets in the build.
Model validation	Prediction accuracy and F1-score within agreed tolerance of the current production model.

7. Deployment Strategy

Validated builds are promoted through three progressively production-like environments. Promotion between environments is gated by automated tests in lower environments and, for the final promotion to Production, by an explicit human approval.



7.1 Environment Details

- Development environment: every merged commit auto-deploys for developer verification and early integration testing; uses synthetic data only.
- Staging/QA environment: a production-like environment used for full regression, load, and UAT testing with anonymized/synthetic student data before sign-off; mirrors production configuration and scale (at reduced capacity).
- Production environment: promoted only after staging sign-off, using a blue-green or canary rollout so live classroom sessions are not disrupted during release.

7.2 Release Strategy Choice

A canary rollout is preferred for the Attendance Capture Service and Prediction Service, since a small percentage of classrooms can validate a new release under real conditions before full rollout. The Instructor Dashboard, which is stateless and low-risk, uses a simpler blue-green swap for faster releases.

8. Security Practices

Because the platform processes biometric identifiers and academic records, security is treated as a first-class, non-negotiable pipeline gate rather than a final review step.

8.1 Data Protection

- Encrypted storage and transmission (TLS 1.2+) of biometric and academic data, with role-based access control for instructors, administrators, and students.
- Field-level encryption for facial embeddings and RFID identifiers; raw images are not retained beyond the matching step.

- Data-retention and deletion policies enforced automatically, aligned with institutional and regulatory requirements.

8.2 Pipeline & Infrastructure Security

- Secrets and credentials managed via a vault service (e.g., HashiCorp Vault / cloud secrets manager) rather than stored in code or pipeline configuration.
- Mandatory security gate: builds with critical vulnerabilities or exposed secrets are automatically blocked from progressing past the code-quality stage.
- Signed, immutable container images with provenance metadata (build id, commit SHA, scan results) attached, preventing untrusted images from being deployed.
- Least-privilege service accounts for each microservice, with network policies restricting inter-service traffic to only what is required.

8.3 Access Control & Audit

- Multi-factor authentication required for anyone approving a production deployment.
- All pipeline runs, approvals, and deployments are logged immutably to support institutional audit and incident investigation.

9. Monitoring, Notifications & Rollback

- Automated health checks (API latency, error rate, model-inference success rate, attendance-capture accuracy) run immediately after each deployment.
- Dashboards (e.g., Grafana) give the DevOps team real-time visibility into each service's health across all three environments.
- Slack/email notifications alert the DevOps and instructor-support teams on pipeline failures or post-deployment anomalies, including the responsible commit and pipeline run link.
- If health checks fail, the pipeline automatically rolls back to the last known-good container image, minimizing downtime for active classrooms, and re-notifies stakeholders once the rollback completes.
- A weekly automated report summarizes deployment frequency, change-failure rate, and mean time to recovery (MTTR) as DevOps performance indicators.

10. Sample Pipeline Configuration (Illustrative)

The following simplified YAML illustrates how the seven stages described above could be expressed as a CI/CD pipeline definition (e.g., GitHub Actions) for one of the platform's microservices.

```
name: smart-classroom-ci-cd
on: [push, pull_request]

jobs:
  build-test-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build Docker image
        run: docker build -t attendance-service:${{ github.sha }} .
      - name: Run unit & integration tests
        run: make test
      - name: Static analysis (SonarQube)
        run: sonar-scanner
      - name: Dependency & secrets scan
        run: snyk test && gitleaks detect
      - name: Model validation (prediction service only)
        run: python validate_model.py --min-accuracy 0.85

  package-and-deploy:
    needs: build-test-scan
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - name: Push image to registry
        run: docker push registry.example.edu/attendance-service:${{ github.sha }}
      - name: Deploy to Development
        run: kubectl apply -f k8s/dev/
      - name: Deploy to Staging (manual approval)
        run: kubectl apply -f k8s/staging/
      - name: Deploy to Production (canary, manual approval)
        run: helm upgrade attendance-service ./charts --set canary.weight=10
```

This configuration is illustrative rather than exhaustive; a production implementation would add caching, matrix builds per microservice, and environment-specific approval rules.

11. Risk Assessment & Mitigation

Risk	Impact	Mitigation
Faulty release disrupts live attendance capture	Instructors cannot take attendance during class.	Canary rollout + automated health-check rollback.
Exposure of biometric or academic data	Privacy breach, regulatory and reputational risk.	Mandatory encryption, access control, and security gate in pipeline.

Risk	Impact	Mitigation
Prediction model drift	At-risk students missed or false alerts flood instructors.	Automated accuracy/drift gate blocking deployment below threshold.
Pipeline outage or misconfiguration	Releases stall; urgent fixes delayed.	Pipeline-as-code under version control with peer review; rollback playbook documented.

12. Roles & Responsibilities

Role	Responsibility in the Pipeline
Developer	Writes code and unit tests, opens pull requests, responds to failed pipeline checks.
Reviewer / Tech Lead	Reviews pull requests, approves merges to main.
DevOps Engineer	Maintains pipeline configuration, environments, and monitoring; owns rollback execution.
Release Owner	Grants manual approval for Staging → Production promotion.
Data Protection Officer	Audits security gate results and data-handling compliance periodically.

13. Pipeline Diagram & Explanation — Code Commit to Deployment

The diagram in Section 4 summarizes the end-to-end flow: a developer commits code to a feature branch and opens a pull request. Once reviewed and merged, the pipeline automatically builds a containerized image, runs the full automated test suite, and performs code-quality and security scanning. Only builds that pass every gate are packaged and pushed to the artifact registry. The image is then deployed first to Development, then promoted to Staging/QA for full validation, and finally released to Production using a controlled rollout strategy. Continuous monitoring after deployment feeds back into the pipeline: any failure triggers an automated rollback to the previous stable release and notifies the responsible team, ensuring the Smart Classroom system remains reliable, secure, and continuously improvable.

14. Conclusion

This document has analyzed the CI/CD requirements of the Smart Classroom Attendance, Engagement, and Performance Analytics System, designed a seven-stage pipeline architecture spanning source control through monitored production deployment, selected and justified tools for each stage, and defined automated testing, security, deployment, and rollback strategies appropriate to a system that handles sensitive student data and supports live classroom operations. Together, these practices ensure that new features and fixes — including improvements to the at-risk prediction model — can be delivered frequently and safely without compromising reliability, security, or the trust of students and instructors who depend on the platform daily.

15. References

- Course material: CSA10 Software Engineering, CO4 – Robust Testing, Quality Assurance, and DevOps Practices.
- Assignment brief: SIMATS Engineering, CO4 Assessment Tool 2 – CI/CD Pipeline Design Exercise, Problem Statement 15 – Smart Classroom Attendance, Engagement, and Performance Analytics System.
- General industry references on CI/CD practice: GitHub Actions documentation, Kubernetes documentation, OWASP Dependency-Check documentation, SonarQube documentation.

End of Submission